

Recommender Systems

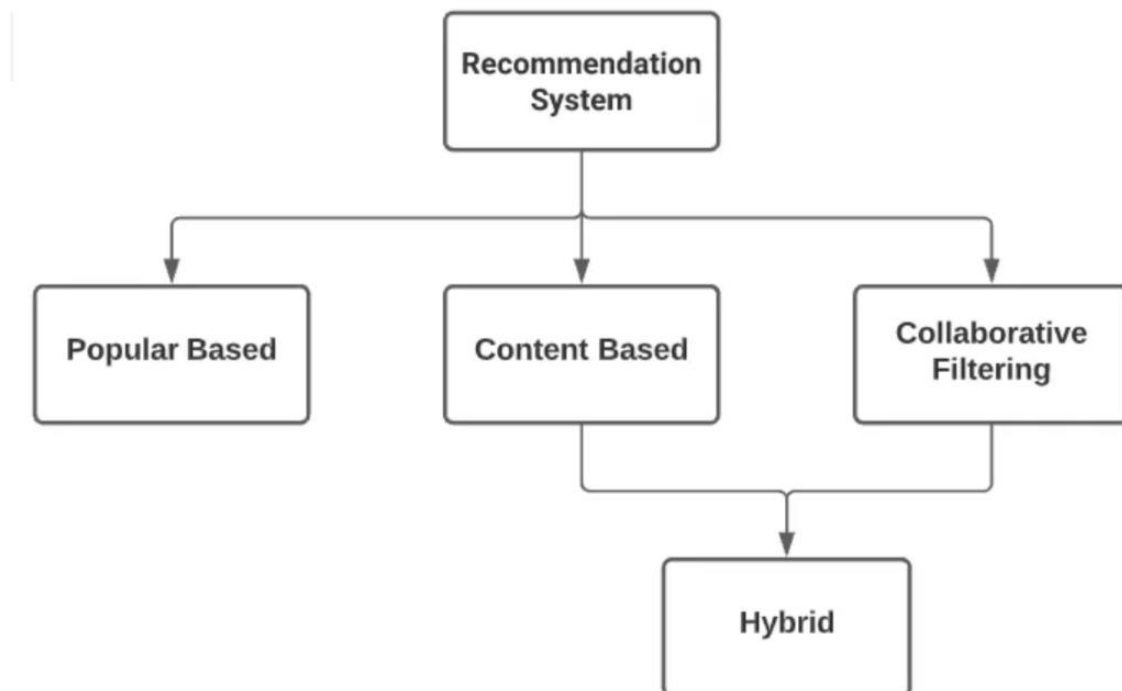
Introduction

A machine learning algorithm known as a recommendation system combines information about users and products to forecast a user's potential interests. These systems are used in a wide range of applications, such as e-commerce, social media, and entertainment, to provide personalized recommendations to users.

There are several types of recommendation systems, including:

1. Popular-based
2. Content-based
3. Collaborative filtering
4. Hybrid

Overview

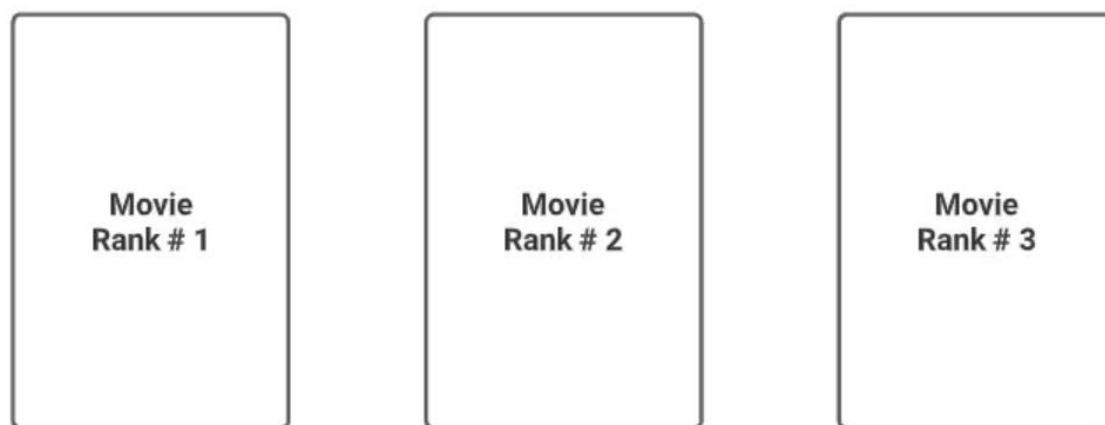


The above figure shows the high-level overview of the recommender system. It looks like it doesn't have many kinds of recommender engines. However, there are many variations within each recommendation based.

1. Popular-based —

This is the baseline performance and the most intuitive recommendation that we can find anywhere. Examples are the **IMDB top-rated movies**, Top 10 in your country today in **Netflix**, etc. These recommendations can be found when you are a new joiner and the provider doesn't have enough information about you. So it would be a safe bet to recommend to you what others like.

Top 10 Series In Your Country Today



Top 10 Series in Country, Image by Author

Limitation :

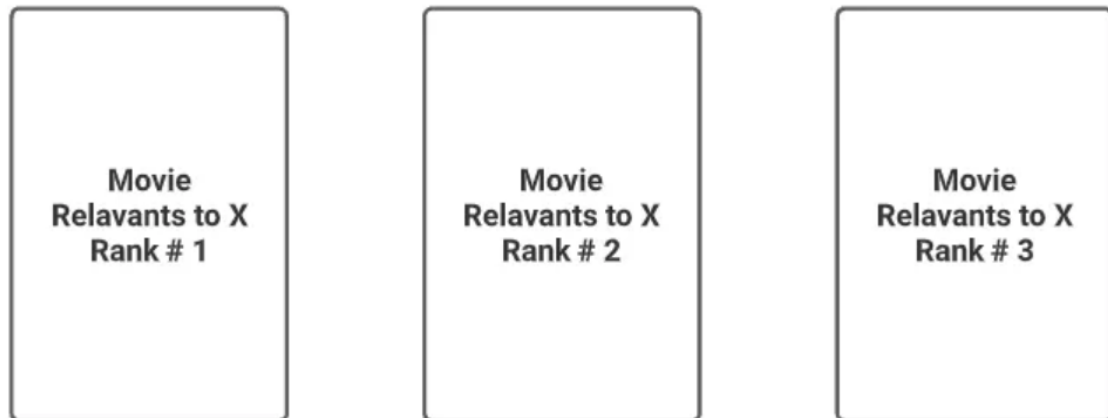
All users get the same recommendation set. It's not personalized.

2. Content-based

—

If you would like to improve the popular-based method, we can incorporate each item's detail to recommend more relevant to the user's desires. For example, you may see the following figure on **Netflix**. It provides you with a list of items because you like some specific movie. The movie would be relevant or similar to what users like in some aspects such as genres, sub-genres, channel, country, etc. The quality of the recommendation depends on how rich your feature set is.

Because you like movie X



Content-based recommendation, Image by author

Limitation:

The recommendation will be limited to what users liked, watched, interacted with before. It doesn't give users a chance to explore a new area they've never been to before. Also, all users who like item X will receive the same recommendation set.

3. Collaborative filtering —

Now, what if you have prior information about the user and the item the user interacted with before. How can you come up with a more sophisticated recommendation engine? This is where collaborative filtering comes to play.

Collaborative filtering recommends the set of items based on what is called the **user-item interaction matrix**. Here is how the user-item interaction matrix look likes.

This type of system uses the past behaviour of users to recommend items that similar users have liked.

Collaborative Filtering

Objective: Recommend Movies to User 3

	User 1	User 2	User 3
Movie 1	5	2	4
Movie 2	3	4	3
Movie 3	1	4	1
Movie 4	2		??
Movie 5	5	2	??

Approach 1: Find Similar Users and recommend movies that they like

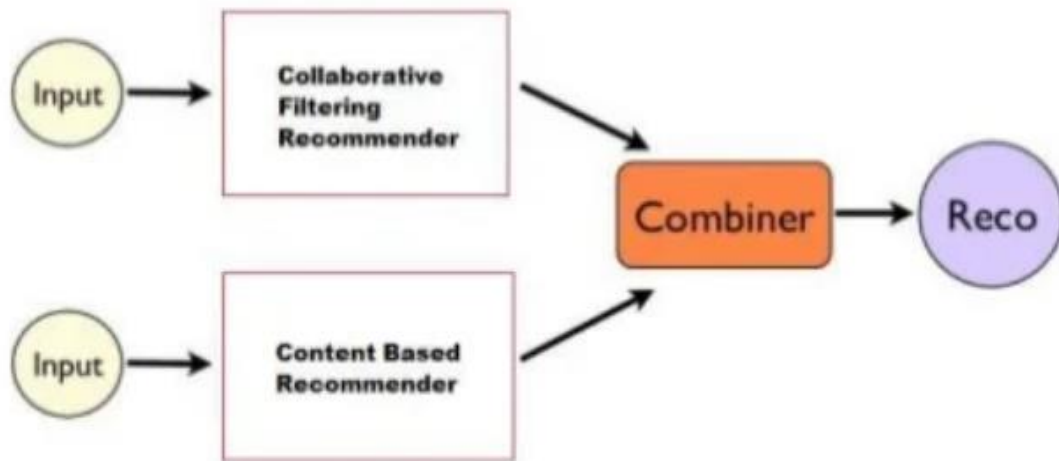
Limitation:

we need prior information about the user before we can derive the similarity for user#1 and others. If you don't have any information about user#1, you can't come up with the prediction because you can't calculate the similarity between users. This problem is called the **Cold-Start** problem. We can fix this problem by providing the new user with the list of **Popular-based** recommendations first and let them score how much they like each famous movie first until we have sufficient information.

4. Hybrid –

We see that each method has its strength. It would be best if we can combine all those strengths and provide a better recommendation. This idea leads us to another improvement of the recommendation, which is the hybrid method. For example, we can combine the content-based and item-based collaborative filtering recommendations together to leverage both domain features (genres and user-item interaction).

Hybrid Approaches



Limitation:

To effectively combine collaborative and content-based methods, a large amount of data is often needed, which might not be available in all contexts.

Movie Recommendation System using content-based filtering

Introduction :

This project demonstrates how to build a Recommendation System that recommends movies based on their content. By utilizing metadata from a CSV file Specifically, this example uses a content-based filtering approach, which focuses on the attributes of the movies (such as genres) to recommend similar movies to users. The results provide personalized movie suggestions based on the content users have previously enjoyed.

Use Case :

- **Increased User Engagement:** Personalized recommendations encourage users to spend more time on the platform, exploring movies that align with their tastes.
- **Higher Satisfaction:** Users are more likely to be satisfied with the service when they receive relevant and enjoyable movie suggestions.
- **Retention and Loyalty:** By providing a personalized experience, the streaming service can improve user retention and foster loyalty.

Code:

```
#importing libraries

import numpy as np
import pandas as pd
import difflib

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

Loading data

movies_data = pd.read_csv(r"C:\Users\purna\Downloads\movies.csv")
movies_data.head()
movies_data.columns
movies_data.shape

# selecting features for recommendation

selected_features = ['genres','keywords','tagline','cast','director']

print(selected_features)

# replacing the null values with null string
```

```
for feature in selected_features:

    movies_data[feature] = movies_data[feature].fillna("")

# combining all the 5 selected features

combined_features = movies_data['genres']+' '+movies_data['keywords']+'
'+movies_data['tagline']+' '+movies_data['cast']+' '+movies_data['director']

# converting the text data to feature vectors

vectorizer = TfidfVectorizer()

feature_vectors = vectorizer.fit_transform(combined_features)

print(feature_vectors)

# getting the similarity scores using cosine similarity

similarity = cosine_similarity(feature_vectors)

print(similarity.shape)

# getting the movie name from the user

movie_name = input(' Enter your favourite movie name : ')

# creating a list with all the movie names given in the dataset

list_of_all_titles = movies_data['title'].tolist()

print(list_of_all_titles)

# finding the close match for the movie name given by the user

find_close_match = difflib.get_close_matches(movie_name, list_of_all_titles)

print(find_close_match)

close_match = find_close_match[0]

print(close_match)

# finding the index of the movie with title
```

```

index_of_the_movie = movies_data[movies_data.title == close_match]['index'].values[0]

print(index_of_the_movie)

# getting a list of similar movies

similarity_score = list(enumerate(similarity[index_of_the_movie]))

print(similarity_score)

# sorting the movies based on their similarity score

sorted_similar_movies = sorted(similarity_score, key = lambda x:x[1], reverse = True)

print(sorted_similar_movies)

# print the name of similar movies based on the index

print('Movies suggested for you : \n')

i = 1

for movie in sorted_similar_movies:

    index = movie[0]

    title_from_index = movies_data[movies_data.index==index]['title'].values[0]

    if (i<30):

        print(i, '.',title_from_index)

        i+=1

```

out put:

```

Movies suggested for you :

1 . Spider-Man
2 . Spider-Man 3
3 . Spider-Man 2
4 . The Notebook
5 . Seabiscuit
6 . Clerks II
7 . The Ice Storm
8 . Oz: The Great and Powerful

```

```
9 . Horrible Bosses
10 . The Count of Monte Cristo
11 . In Good Company
12 . Finding Nemo
13 . Clear and Present Danger
14 . Brothers
15 . The Good German
16 . Drag Me to Hell
17 . Bambi
18 . The Queen
19 . Charly
20 . Escape from L.A.
21 . Daybreakers
22 . The Life Aquatic with Steve Zissou
23 . Labor Day
24 . Wimbledon
25 . Cold Mountain
26 . Hearts in Atlantis
27 . Out of the Furnace
28 . Bullets Over Broadway
29 . The Purge: Election Year
```

Movie Recommendation System using Collaborative-based filtering

Introduction :

This project demonstrates how to build a Recommendation System using collaborative filtering based on user ratings. In this approach, we use a dataset containing user IDs, movie IDs, and ratings to suggest movies to users. By analyzing how different users rate movies, we can identify patterns and similarities in preferences. This allows us to recommend movies that users with similar tastes have enjoyed, providing personalized suggestions based on the collective behavior of the user base.

Use Case :

- **More Engagement:** Users are encouraged to explore more movies because the recommendations are tailored to their preferences.
- **Greater Satisfaction:** Users are happier with movie suggestions that match their tastes, leading to a better overall experience.
- **Increased Loyalty:** Personalized recommendations keep users coming back, as they find the platform more relevant and enjoyable.

Code:

```
import pandas as pd

from scipy import sparse

ratings = pd.read_csv(r"C:\Users\purna\Downloads\ratings.csv")
```

```

movies = pd.read_csv(r"C:\Users\purna\Downloads\movies (1).csv")
ratings = pd.merge(movies, ratings).drop(['genres', 'timestamp'], axis=1)
print(ratings.shape)
ratings.head()

userRatings = ratings.pivot_table(index=['userId'],columns=['title'],values='rating')
userRatings.head()

print("Before: ",userRatings.shape)

userRatings = userRatings.dropna(thresh=10, axis=1).fillna(0,axis=1)
#userRatings.fillna(0, inplace=True)

print("After: ",userRatings.shape)

corrMatrix = userRatings.corr(method='pearson')
corrMatrix.head(100)

def get_similar(movie_name,rating):

    similar_ratings = corrMatrix[movie_name]*(rating-2.5)

    similar_ratings = similar_ratings.sort_values(ascending=False)

    return similar_ratings

romantic_lover = [("500) Days of Summer (2009)",5),("Alice in Wonderland (2010)",3),("Aliens (1986)",1),("2001: A Space Odyssey (1968)",2)]

similar_movies = pd.DataFrame()

for movie,rating in romantic_lover:

    similar_movies = similar_movies.append(get_similar(movie,rating),ignore_index = True)

similar_movies.head(10)

similar_movies.sum().sort_values(ascending=False).head(20)

action = [("Amazing Spider-Man, The (2012)",5),("Mission: Impossible III (2006)",4),("Toy Story 3 (2010)",2),("2 Fast 2 Furious (Fast and the Furious 2, The) (2003)",4)]

similar_movies = pd.DataFrame()

for movie,rating in action:

    similar_movies = similar_movies.append(get_similar(movie,rating),ignore_index = True)

```

```
similar_movies.head(10)
```

```
similar_movies.sum().sort_values(ascending=False).head(10)
```

out put

```
title
Amazing Spider-Man, The (2012)          3.233134
Mission: Impossible III (2006)          2.874798
2 Fast 2 Furious (Fast and the Furious 2, The) (2003)  2.701477
Over the Hedge (2006)                  2.229721
Crank (2006)                           2.176259
Mission: Impossible - Ghost Protocol (2011)  2.159666
Hancock (2008)                         2.156098
The Amazing Spider-Man 2 (2014)         2.153677
Hellboy (2004)                         2.137518
Snakes on a Plane (2006)               2.137396
dtype: float64
```

Movie Recommendation System using Hybrid filtering

Introduction :

This project illustrates the creation of a Recommendation System for fashion products using a hybrid recommendation approach. By combining both content-based filtering and collaborative filtering techniques, this system leverages product attributes (such as style and brand) as well as user preferences and behavior to recommend fashion items. The content-based component focuses on the features of the products, while the collaborative filtering component uses user interactions and preferences. The result is a sophisticated recommendation system that offers personalized fashion product suggestions tailored to each user's tastes and browsing history.

Use Case:

- **Improved User Engagement:** By integrating both content-based and collaborative filtering, users receive fashion recommendations that are both relevant to their style preferences and influenced by similar users' choices, encouraging them to explore more products.
- **Enhanced Satisfaction:** The hybrid approach ensures that users get recommendations that match their tastes and preferences, leading to a more satisfying shopping experience.
- **Increased Conversion Rates:** Personalized suggestions based on a combination of product attributes and user behavior can lead to higher conversion rates, as users are more likely to purchase items that align with their preferences and those liked by similar users.

.

Code:

```
import pandas as pd
```

```

data = pd.read_csv(r"C:\Users\purna\Downloads\fashion_products.csv")
data.head(10)
data.isnull().sum()

pip install scikit-surprise

hybrid(content+collaborative)

1.content based-> movie, music, books
2.collaborative filtering-> user behaviour, pattern, rating

from surprise import Dataset, Reader, SVD
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel

# Content-Based Filtering

content_df = data[['Product ID', 'Product Name', 'Brand',
                  'Category', 'Color', 'Size']]

content_df['Content'] = content_df.apply(lambda row: ' '.join(row.dropna().astype(str)),
axis=1)

# Use TF-IDF vectorizer to convert content into a matrix of TF-IDF features

tfidf_vectorizer = TfidfVectorizer()

content_matrix = tfidf_vectorizer.fit_transform(content_df['Content'])

content_similarity = linear_kernel(content_matrix, content_matrix)

reader = Reader(rating_scale=(1, 5))

data = Dataset.load_from_df(data[['User ID',
                                'Product ID',
                                'Rating']], reader)

def get_content_based_recommendations(product_id, top_n):
    index = content_df[content_df['Product ID'] == product_id].index[0]
    similarity_scores = content_similarity[index]

```

```

similar_indices = similarity_scores.argsort()[::-1][1:top_n + 1]
recommendations = content_df.loc[similar_indices, 'Product ID'].values
return recommendations

# Collaborative Filtering
algo = SVD()
trainset = data.build_full_trainset()
algo.fit(trainset)

def get_collaborative_filtering_recommendations(user_id, top_n):
    testset = trainset.build_anti_testset()
    testset = filter(lambda x: x[0] == user_id, testset)
    predictions = algo.test(testset)
    predictions.sort(key=lambda x: x.est, reverse=True)
    recommendations = [prediction.iid for prediction in predictions[:top_n]]
    return recommendations

# Hybrid Approach
def get_hybrid_recommendations(user_id, product_id, top_n):
    content_based_recommendations = get_content_based_recommendations(product_id,
top_n)

    collaborative_filtering_recommendations =
get_collaborative_filtering_recommendations(user_id, top_n)

    hybrid_recommendations = list(set(content_based_recommendations +
collaborative_filtering_recommendations))

    return hybrid_recommendations[:top_n]

```

```
user_id = 79

product_id = 5

top_n = 10

recommendations = get_hybrid_recommendations(user_id, product_id, top_n)

print(f"Hybrid Recommendations for User {user_id} based on Product {product_id}:")

for i, recommendation in enumerate(recommendations):

    print(f"{i + 1}. Product ID: {recommendation}")
```

out put :

```
Hybrid Recommendations for User 79 based on Product 5:
1. Product ID: 1700
2. Product ID: 837
3. Product ID: 683
4. Product ID: 1387
5. Product ID: 1558
6. Product ID: 1372
7. Product ID: 952
8. Product ID: 1018
9. Product ID: 1532
10. Product ID: 509
```

